

Systeme de Prediction de Defaut de Paiement

Rapport Complet du Pipeline de Machine Learning

Auteur : Yomna Haouel

Projet : Data Science - Credit Scoring

Dataset : Home Credit Default Risk

*Pipeline complet : EDA, Preprocessing, Feature Engineering,
Feature Selection, Multi-Model Training, MLflow, SHAP, FastAPI*

Table des Matieres

1. Vue d'Ensemble de l'Architecture
2. Le Dataset - Home Credit Default Risk
3. Etape 0 - Analyse Exploratoire (EDA)
4. Etape 1 - Preprocessing Anti-Fuite
5. Etape 2 - Feature Engineering
6. Etape 3 - Selection de Variables
7. Etape 4 - Entrainement Multi-Modeles
8. Etape 5 - Comparaison et Selection du Meilleur Modele
9. Deploiement - API FastAPI
10. Bonnes Pratiques Implementees
11. Instructions d'Execution
12. Points Cles pour la Soutenance

1. Vue d'Ensemble de l'Architecture

Le projet suit une architecture modulaire industrielle : chaque etape du pipeline est isolee dans un fichier Python dedie, et tous les parametres sont centralises dans un fichier de configuration unique (config.py). L'orchestration est assuree par code/main.py qui enchaîne les 5 etapes dans l'ordre.

Structure des fichiers

```
config.py          -> Configuration centralisee
code/main.py       -> Orchestrateur principal (CLI)
code/exploration_donnees.py -> Etape 0 : EDA
code/preprocessing.py -> Etape 1 : Nettoyage anti-fuite
code/feature_engineering.py -> Etape 2 : Creation de variables
code/feature_selection.py -> Etape 3 : Selection de variables
code/training.py   -> Etape 4 : Entrainement + MLflow
code/model_comparison.py -> Etape 5 : Comparaison + SHAP
api/app.py         -> Deploiement FastAPI
```

Pourquoi cette architecture ?

- * Reproductibilite : tout parametre est dans config.py, jamais en dur dans le code
- * Maintenabilite : modifier une etape n'affecte pas les autres
- * Testabilite : chaque module peut etre execute/teste independamment
- * Tracabilite : MLflow enregistre chaque experience

2. Le Dataset - Home Credit Default Risk

Caracteristique	Valeur
Fichier	feature_matrix.csv
Dimensions	356 255 lignes, 1 698 colonnes
Train	307 511 lignes (set = 'train')
Test	48 744 lignes (set = 'test')
Variable cible	TARGET : 0=bon payeur, 1=default
Desequilibre	~8% de defaults (classe 1)

Contexte metier : Une institution financiere doit decider si un client va rembourser son credit. Le cout d'un defaut non detecte (faux negatif) est bien superieur au cout d'un refus a tort (faux positif). C'est pourquoi le Recall est une metrique prioritaire.

Le dataset est fortement desequilibre (~8% de defaults). Sans traitement adapte, un modele naif predisant toujours '0' obtiendrait 92% d'accuracy mais 0% de Recall - completement inutile en pratique.

3. Etape 0 - Analyse Exploratoire (EDA)

Fichier : `code/exploration_donnees.py`

Commande : `python code/main.py --eda-only`

3.1 Distribution de la cible

La fonction `analyze_target()` calcule le taux de défaut (~8.07%) et genere un diagramme en barres. Cette analyse revele le desequilibre des classes qui necessitera un traitement special (SMOTE) lors de l'entrainement.

3.2 Analyse des valeurs manquantes

La fonction `analyze_missing()` identifie les colonnes avec plus de 50% de valeurs manquantes et genere un histogramme des taux de missing. Cette analyse justifie le seuil de suppression a 60% utilise dans le preprocessing.

3.3 Distributions numeriques

La fonction `analyze_numeric()` trace les histogrammes des 20 variables numeriques les plus correlees avec TARGET, separees par classe (default vs non-default). Cela permet d'identifier visuellement les variables discriminantes.

3.4 Detection d'outliers

Methode IQR (InterQuartile Range) : un point est considere comme outlier s'il est en dehors de $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$. La fonction `detect_outliers()` identifie les colonnes les plus polluees par les valeurs aberrantes et genere un Top 20 des colonnes avec le plus d'outliers.

3.5 Analyse categorielle

Distribution des variables categorielles et taux de default par modalite. Tres utile pour des variables comme NAME_CONTRACT_TYPE, CODE_GENDER, etc.

3.6 Correlation avec la cible

Top 20 des variables les plus correlees avec TARGET + heatmap de correlation. Donne un premier apercu des variables importantes (EXT_SOURCE_1/2/3, DAYS_BIRTH, etc.).

3.7 Recommandations business

5 insights automatiques generes a la fin de l'EDA : gestion des missing, traitement du desequilibre, variables cles, feature engineering suggere, strategie de modelisation.

Pourquoi l'EDA est essentielle ? L'EDA guide toutes les decisions en aval : quels seuils de nettoyage choisir, quelles variables transformer, quel algorithme privilegier. C'est la fondation scientifique du projet.

4. Etape 1 - Preprocessing Anti-Fuite

Fichier : code/preprocessing.py - Classe : DataPreprocessor

4.1 Chargement et filtrage

La methode `load_and_filter()` garde uniquement les lignes d'entrainement (`set == 'train'`), supprime les colonnes d'identification (`SK_ID_CURR`, `set`), et separe X (features) et y (TARGET).

4.2 Nettoyage basique

La methode `basic_cleaning()` supprime les colonnes avec plus de 60% de valeurs manquantes (829 colonnes supprimees sur 1 695) et supprime les colonnes a variance nulle (constantes). Resultat : 867 colonnes restantes.

4.3 Split stratifie

`train_test_split(test_size=0.2, stratify=y, random_state=42)` produit 246 008 echantillons d'entrainement et 61 503 de validation. Le split stratifie preserve le ratio ~8% de defauts dans les deux ensembles.

4.4 Pipeline sklearn (ColumnTransformer)

Un `ColumnTransformer` avec deux branches paralleles :

- * Numeriques : `SimpleImputer(strategy='median')` puis `StandardScaler()`
- * Categorieelles : `SimpleImputer(strategy='most_frequent')` puis `OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1)`

4.5 Fit-Transform anti-fuite

`fit_transform()` est applique sur le train UNIQUEMENT. `transform()` est applique sur le test (jamais de fit sur le test !). Le pipeline est sauvegarde (`preprocessing_pipeline.joblib`) pour le deploiement en production.

POURQUOI L'ANTI-FUITE EST CRITIQUE ?

La fuite de donnees (data leakage) est l'erreur la plus grave en ML : si le modele 'voit' des informations du test pendant l'entrainement, les metriques sont artificiellement gonflees et le modele ne generalisera pas en production.

Notre pipeline garantit ZERO fuite :

- Le split est fait AVANT toute transformation
- Le `StandardScaler` apprend la moyenne/ecart-type sur le train uniquement
- L'`OrdinalEncoder` apprend les categories sur le train uniquement
- Le SMOTE (etape 4) est applique sur le train uniquement

5. Etape 2 - Feature Engineering

Fichier : `code/feature_engineering.py` - Classe : `FeatureEngineer`

20 nouvelles variables creees, regroupees en 5 familles :

5.1 Ratios financiers (6 variables)

Variable	Formule	Logique Metier
CREDIT_INCOME_RATIO	Credit / Revenu	Capacite de remboursement
ANNUITY_INCOME_RATIO	Annuite / Revenu	Poids des mensualites
CREDIT_TERM	Credit / Annuite	Duree estimee du credit
CREDIT_GOODS_RATIO	Credit / Prix du bien	Surcout du credit
INCOME_PER_PERSON	Revenu / Nb famille	Revenu par personne
ANNUITY_CREDIT_RATIO	Annuite / Credit	Taux de remboursement

5.2 Age et Emploi (4 variables)

Variable	Description
AGE_YEARS	Age en annees (DAYS_BIRTH / -365)
EMPLOYED_ANOMALY	Flag binaire : DAYS_EMPLOYED == 365243 (anomalie connue)
EMPLOYMENT_YEARS	Anciennete en annees (DAYS_EMPLOYED / -365)
EMPLOYMENT_AGE_RATIO	Ratio anciennete / age

La valeur 365243 dans DAYS_EMPLOYED est une anomalie connue du dataset Home Credit (retraites, chomeurs). Un flag binaire EMPLOYED_ANOMALY est cree pour la capturer explicitement.

5.3 Interactions EXT_SOURCE (7 variables)

Les scores externes (EXT_SOURCE_1, EXT_SOURCE_2, EXT_SOURCE_3) proviennent de bureaux de credit et sont les variables les plus predictives du dataset. Les interactions creees sont :

- * Statistiques agregees : EXT_SOURCE_MEAN, EXT_SOURCE_STD, EXT_SOURCE_MIN, EXT_SOURCE_MAX
- * Produits croises : EXT1 x EXT2, EXT1 x EXT3, EXT2 x EXT3

Ces interactions capturent des relations non-lineaires que les modeles lineaires ne capturent pas seuls.

5.4 Documents fournis (1 variable)

DOCUMENTS_PROVIDED_COUNT = somme des 20 flags FLAG_DOCUMENT_*. Un client qui fournit plus de documents est souvent plus solvable.

5.5 Variables sociales (2 variables)

- * REGION_MISMATCH_SUM : somme des incoherences geographiques (client/employeur/contact)
- * CONTACT_INFO_COUNT : nombre de moyens de contact fournis (signaux de fraude potentielle)

Pourquoi le Feature Engineering est important ?

- Les algorithmes ML travaillent mieux avec des combinaisons significatives qu'avec des variables brutes
- CREDIT_INCOME_RATIO est bien plus discriminant que AMT_CREDIT et AMT_INCOME_TOTAL pris separement
- Les interactions EXT_SOURCE capturent des synergies entre scores de credit
- Gestion robuste : verification d'existence des colonnes sources, remplacement des divisions par zero

6. Etape 3 - Selection de Variables

Fichier : code/feature_selection.py - Classe : FeatureSelector

Processus en 3 phases + strategie de fusion

Phase 1 - Filtre de correlation

Calcule la matrice de correlation de Pearson (sur un echantillon de 50K lignes pour la performance). Si deux variables ont $|correlation| > 0.85$, celle avec la plus faible correlation avec TARGET est supprimee.

Resultat : 850 -> 449 variables (401 redondantes supprimees)

Pourquoi ? La multicolinearite degrade les modeles lineaires et alourdit les modeles d'ensemble sans gain predictif.

Phase 2 - Information Mutuelle

Calcule le MI score de chaque variable avec TARGET (sur un echantillon de 80K). Garde les top 120 variables (parametre MI_TOP_K). L'information mutuelle capture les dependances NON-LINEAIRES, contrairement a la correlation de Pearson.

Pourquoi ? Identifie des variables importantes que la correlation lineaire pourrait manquer.

Phase 3 - Importance par arbre (LightGBM)

Entraîne un LightGBM rapide et extrait l'importance des variables (feature_importances_). Garde les variables dont l'importance normalisee > 0.001 . 230 variables retenues.

Pourquoi ? Les modeles d'ensemble evaluent l'utilite réelle de chaque variable dans des splits de decision.

Fusion - Strategie Union

Notre choix : la strategie UNION garde toute variable selectionnee par AU MOINS une methode. L'alternative 'intersection' ne garderait que les variables selectionnees par TOUTES les methodes (plus restrictif).

Resultat final : de 886 variables -> 269 variables finales (reduction de 70%)

Pourquoi union ? Chaque methode a des forces differentes (lineaire vs non-lineaire vs importance d'arbre). L'union maximise les chances de garder les bonnes variables. Les features selectionnees sont sauvegardees dans selected_features.json pour le deploiement.

7. Etape 4 - Entrainement Multi-Modeles

Fichier : code/training.py

7.1 SMOTE (Synthetic Minority Oversampling)

Le dataset est desequilibre (8% de defaults). Sans traitement, les modeles predisent '0' tout le temps et obtiennent 92% d'accuracy... mais 0% de Recall !

SMOTE genere des echantillons synthetiques de la classe minoritaire par interpolation entre voisins KNN. Avec `sampling_strategy=0.5`, la classe minoritaire atteint 50% de la classe majoritaire apres SMOTE (ratio 1:2 au lieu de 1:12).

SMOTE est applique sur le train UNIQUEMENT - jamais sur le test. C'est une regle fondamentale de l'anti-leakage.

7.2 Les 5 modeles entraines

Modele	Type	Particularite
LogisticRegression	Lineaire	<code>class_weight='balanced', C=0.01</code>
RandomForest	Bagging	200 arbres, prof. 12, balanced
XGBoost	Boosting	300 arbres, lr=0.03, scale_pos_weight=10
LightGBM	Boosting	500 arbres, lr=0.03, is_unbalance=True
CatBoost	Boosting	500 iter, lr=0.03, auto_class_weights

7.3 Metriques d'evaluation

Chaque modele est evalue sur le jeu de validation (jamais vu pendant l'entrainement) :

- * ROC-AUC : mesure la capacite discriminante globale du modele (independante du seuil)
- * Recall : proportion de vrais defaults detectes (priorite business)
- * Precision : proportion de predictions 'defaut' correctes
- * F1-Score : moyenne harmonique precision/recall

Pour chaque modele, une matrice de confusion et une courbe ROC sont generees et sauvegardees dans `plots/training/`.

7.4 Tracking MLflow

Chaque run MLflow enregistre : les parametres du modele, les metriques de performance, et le modele serialise. Cela permet de comparer les experiences a posteriori, de retrouver les meilleurs hyperparametres, et de reproduire n'importe quel resultat.

8. Etape 5 - Comparaison et Selection

Fichier : code/model_comparison.py

8.1 Tableau comparatif des resultats

Modele	ROC-AUC	Recall	F1-Score	Temps (s)
LightGBM	0.7771	0.1253	0.1947	29.8
CatBoost	0.7738	0.1148	0.1815	81.2
XGBoost	0.7583	0.7140	0.2598	25.1
RandomForest	0.7375	0.1732	0.2125	107.0
LogisticRegression	0.6519	0.6137	0.2034	78.7

8.2 Critere de selection pondere

La formule de selection du meilleur modele est :

$$\text{Score} = 0.50 \times \text{AUC} + 0.30 \times \text{Recall} + 0.20 \times \text{F1}$$

Ponderation :

- 50% AUC : pouvoir discriminant global
- 30% Recall : detection des defauts (objectif metier prioritaire)
- 20% F1 : equilibre precision/recall

RESULTAT : XGBoost gagne avec un score pondere de 0.6453.

Justification : XGBoost offre le meilleur compromis AUC (0.7583) + Recall eleve (0.7140). LightGBM a un meilleur AUC (0.7771) mais un Recall tres faible (0.1253), ce qui signifie qu'il ne detecte que 1 defaut sur 8 - inacceptable en contexte bancaire ou le cout d'un defaut non detecte est tres superieur au cout d'un faux positif.

8.3 Analyse SHAP (Interpretabilite)

TreeExplainer de SHAP calcule la contribution marginale de chaque variable a chaque prediction. Le summary plot montre les 20 variables les plus influentes du meilleur modele (XGBoost). Cette interpretabilite est indispensable en contexte bancaire pour la conformite reglementaire et la transparence des decisions de credit.

9. Deploiement - API FastAPI

Fichier : `api/app.py`

L'API REST permet de deployer le modele en production pour des predictions en temps reel. Elle charge au demarrage le meilleur modele (`best_model.joblib`), la liste des features (`selected_features.json`), et le pipeline de preprocessing.

Endpoints disponibles

Endpoint	Methode	Description
/	GET	Health check - verifie que le modele est charge
/info	GET	Informations du modele et features attendues
/predict	POST	Prediction pour un client unique

Classification du risque

Probabilite	Niveau de Risque
< 0.2	LOW (Faible)
0.2 a 0.5	MEDIUM (Moyen)
0.5 a 0.8	HIGH (Eleve)
>= 0.8	VERY HIGH (Tres Eleve)

Exemple d'utilisation

```
# Lancer le serveur :
uvicorn api.app:app --host 0.0.0.0 --port 8000 --reload

# Envoyer une requete de prediction :
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"features": {"AMT_CREDIT": 500000, ...}}'
```

La reponse contient : la probabilite de defaut, la prediction binaire (0 ou 1), le niveau de risque (LOW/MEDIUM/HIGH/VERY HIGH), et le seuil utilise (0.5).

10. Bonnes Pratiques Implementees

Pratique	Implementation	Justification
Anti data leakage	Split avant transformation	Metriques fiables
Config centralisee	Tout dans config.py	Reproductibilite
Architecture modulaire	1 fichier = 1 responsabilite	Maintenabilite
SMOTE train only	Jamais sur le test	Pas de contamination
Selection multi-methodes	Corr + MI + Tree	Robustesse
Tracking MLflow	Params + metriques + modele	Tracabilite
Seuils ponderes	AUC/Recall/F1	Choix objectif
SHAP	Interpretabilite post-hoc	Conformite reglementaire
API REST	FastAPI + Pydantic	Production-ready

11. Instructions d'Execution

Installation

```
pip install -r requirements.txt
```

Lancer l'EDA seule

```
python code/main.py --eda-only
```

Lancer le pipeline complet (avec EDA)

```
python code/main.py --eda
```

Lancer le pipeline complet (sans EDA, plus rapide)

```
python code/main.py
```

Lancer l'API de prediction

```
uvicorn api.app:app --host 0.0.0.0 --port 8000 --reload
```

Visualiser les experiences MLflow

```
mlflow ui --port 5000
```

12. Points Cles pour la Soutenance

Question : Pourquoi XGBoost est le meilleur modele ?

XGBoost offre le meilleur compromis entre AUC (0.7583) et Recall (0.7140). Bien que LightGBM ait un AUC superieur (0.7771), son Recall de 12.5% signifie qu'il ne detecte que 1 defaut sur 8 - ce qui est inacceptable dans un contexte de risque de credit ou le cout d'un defaut non detecte est tres superieur au cout d'un faux positif.

Question : Comment avez-vous gere le desequilibre ?

Trois niveaux :

- 1) SMOTE pour reequilibrer le train a un ratio 1:2
- 2) Hyperparametres natifs (scale_pos_weight, class_weight='balanced', is_unbalance)
- 3) Optimisation sur le Recall et l'AUC plutot que l'accuracy

Question : Comment garantisiez-vous l'absence de data leakage ?

Le split intervient AVANT toute transformation. Le pipeline sklearn est fit sur le train uniquement puis transform sur le test. Le SMOTE est applique au train uniquement. Aucune information du test ne 'fuit' dans l'entrainement.

Question : Pourquoi 3 methodes de selection de variables ?

Chaque methode capture des aspects differents :

- La correlation filtre la redondance lineaire
- L'information mutuelle detecte les dependances non-lineaires
- L'importance d'arbre evalue l'utilite reelle dans un modele de decision

L'union des trois garantit qu'aucune variable utile n'est perdue.

Question : Pourquoi utiliser MLflow ?

MLflow assure la tracabilite complete des experiences : chaque run enregistre les hyperparametres, les metriques et le modele serialise. Cela permet de reproduire n'importe quel resultat, de comparer les experiences a posteriori, et de deployer facilement le meilleur modele en production.

Question : Comment le modele est-il deploye ?

Via une API REST FastAPI. Au demarrage, l'API charge le best_model.joblib et le selected_features.json. Un endpoint POST /predict recoit les donnees client en JSON, aligne les features, effectue la prediction, et retourne la probabilite de defaut avec un niveau de risque (LOW/MEDIUM/HIGH/VERY HIGH).